

PEARSON SPECTER LITT

Legal Draft Assistant

*Building AI That Thinks Like a Lawyer, Built by Someone Who Thinks Like
a Founder*

WHITE PAPER

Shihab Alam Jamshed · May 2026

AI Engineer Take-Home Assessment

CONTENTS

01 Executive Summary

02 The Problem: What Messy Legal Documents Actually Cost

03 What the System Does

- Ingestion & OCR Pipeline
- Retrieval & Evidence Grounding
- Draft Generation
- The Learning Loop

04 Why a Legal Team Needs This

05 The Founder's Journey: Research to Execution

- Starting from Zero
- What ChatGPT Taught Me
- The Budget & Time Reality
- The Decisions I Made Under Pressure

06 Architecture & Technology Stack

07 What Was Built vs. What Was Omitted — and Why

08 The Path to Production

09 Conclusion

01

Executive Summary

Legal work runs on documents. Contracts, eviction notices, inspection reports, lease agreements — every matter leaves a paper trail. The problem is that in the real world, those documents are rarely clean. They are scanned at odd angles, faxed three times before arriving, handwritten on lined paper, or exported from legacy systems that strip formatting with abandon. A junior paralegal can spend hours extracting the facts a senior attorney needs in five minutes.

The Legal Draft Assistant is a Python-based AI pipeline built to close that gap. It ingests messy legal documents of any quality, extracts structured evidence, retrieves the most relevant facts, and generates Case Fact Summary Memos that cite their sources — making every claim traceable back to a specific passage in the original document. When a lawyer edits the output, the system learns from those corrections and applies them to every future draft automatically.

This white paper tells the full story: what the system does, why a legal team would need it, and — in a separate section — the honest account of how it was conceived, researched, constrained by a 12-hour deadline and a zero-dollar budget, and shipped anyway.

"I optimized for grounded, inspectable workflows under tight constraints rather than overbuilding infrastructure."

The Problem: What Messy Legal Documents Actually Cost

Before writing a single line of code, the first question was: what does 'messy legal document' actually mean in practice? The answer turned out to be more layered than expected.

A messy document is not just a bad scan. It is a document where the signal — the facts that matter to a lawyer — is buried under noise: OCR artifacts, misaligned columns, faded ink, inconsistent formatting across ten different firms' letterheads. The problem compounds: an attorney reviewing an eviction case might deal with a handwritten inspection note, a digitally signed lease, a scanned notice of default, and a faxed payment record — all in the same matter, all in different formats.

The cost is real and measurable:

Pain Point	Impact
Manual document review	2–4 hours per matter for a paralegal
Inconsistent memo structure	Rework and version confusion
Undiscovered facts in poor scans	Missed evidence, legal exposure
No institutional memory of edits	Same mistakes repeated across matters
AI tools that hallucinate	Unacceptable risk in legal context

The legal industry is not short on AI tools. It is short on AI tools that are trustworthy enough to use in a professional context — tools where every generated claim can be traced to a source, where uncertainty is acknowledged rather than papered over, and where the system gets better the more it is used. That gap is what this system was built to fill.

What the System Does

The system is a four-stage pipeline. Each stage has one job and passes clean output to the next. There is no magic — every step is explicit, inspectable, and replaceable.

Upload PDF or Image
OCR + Text Extraction (Tier 1: pymupdf Tier 2: Tesseract Tier 3: Image OCR)
Cleaning + Chunking (600-char chunks, 100-char overlap, confidence scored)
Metadata Extraction (parties, dates, addresses, document type)
Embeddings + FAISS Index (sentence-transformers, cosine similarity)
Grounded Retrieval (top-k chunks, confidence-weighted relevance)
Draft Generation (evidence-only prompting, every claim cites a chunk ID)
Operator Edit → Pattern Extraction → Correction Memory
Improved Future Drafts (learned preferences injected into system prompt)

Stage 1: Ingestion & OCR Pipeline

The system accepts any PDF or image file. Rather than running full OCR on everything — which is slow and unnecessary — it uses a tiered approach. Digital PDFs are parsed directly with pymupdf, which is fast and produces clean text with 100% confidence. If a page yields fewer than 50 characters (a sign it is a scanned image embedded in a PDF), the system falls back to Tesseract OCR. Raw image files go to Tesseract directly.

Every chunk of text carries an OCR confidence score from 0.0 to 1.0. This score follows the text through the entire pipeline — it is not discarded after extraction. Low-confidence chunks are not excluded; they are penalised in retrieval scoring so that clean evidence is always ranked above noisy evidence of equal semantic similarity.

Stage 2: Retrieval & Evidence Grounding

The extracted text is split into overlapping 600-character chunks and embedded using sentence-transformers (the all-MiniLM-L6-v2 model, which runs locally at no API cost). These embeddings are stored in a FAISS index — a lightweight vector database that requires no server and produces inspectable local files.

When a draft is requested, the system retrieves the top-k most relevant chunks using a confidence-weighted relevance score:

```
relevance = cosine_similarity × (1.0 if confidence ≥ 0.3 else 0.5)
```

This formula means a highly relevant but poorly-OCR'd chunk will lose to a slightly less similar but clean chunk. Generation quality is bounded by evidence quality — by design.

Stage 3: Draft Generation

This is where the system differs most sharply from generic AI tools. The language model is given only the retrieved evidence chunks — nothing else. The system prompt enforces three hard rules: use only the provided evidence, cite every claim with a chunk ID, and explicitly say [UNCERTAIN] or [NOT FOUND IN DOCUMENTS] rather than filling gaps with plausible-sounding fabrications.

The result is a Case Fact Summary Memo — a structured document listing parties, key dates, material facts, potential issues, and supporting evidence — where every sentence that makes a factual claim carries a bracketed reference like [eviction_notice_chunk_0004]. A reviewer can trace any claim back to the original text in seconds. Hallucinations do not disappear silently; they leave a visible absence of citations.

Stage 4: The Learning Loop

Most AI legal tools stop at generation. This system has a fourth stage: when an operator edits a draft, the system captures both versions, sends them to the language model with the question 'what reusable pattern can be learned from this edit?', and stores the result as an abstract rule in a correction memory file.

These are not per-document overrides. They are generalised preferences — rules like 'always include a chronological timeline when more than two key dates are present' or 'never make intent claims unsupported by evidence' — that are injected into the system prompt for every future draft. The system gets measurably better with use, without any retraining or fine-tuning.

Why a Legal Team Needs This

The legal profession has a trust problem with AI. It is not that lawyers distrust technology — it is that the stakes of a hallucinated fact are categorically different in a legal brief than in a marketing email. A tool that confidently invents a clause, a date, or a party name is not just unhelpful; it is a liability.

This system was designed with that reality as the primary constraint. Every architectural decision traces back to a single question: can a lawyer defend this output in court? The answer has to be yes — and the way to make it yes is to make every claim auditable.

Why It Matters	What the System Delivers
Document review takes hours	Automated ingestion + structured memo in seconds
Scanned documents miss evidence	Three-tier OCR catches text others miss
AI tools hallucinate	Every claim cites a source; gaps are flagged explicitly
Attorneys edit outputs differently	Learning loop captures preferences and applies them firm-wide
Budgets and timelines are real	No database, no cloud infra, no per-query embedding cost
Junior staff make inconsistent memos	Standardised output structure enforced by the system prompt

There is also a broader point about where AI fits in a legal workflow. This system is not trying to replace legal reasoning. It is trying to eliminate the grunt work that precedes it — so that the attorney walks into a meeting with a structured memo and cited evidence already in hand, rather than a stack of unprocessed documents.

The correction memory feature is where the long-term value compounds. Every firm has institutional knowledge about how memos should be structured for their practice areas, their judges, their clients. That knowledge lives in the heads of senior attorneys and gets lost when they leave. This system captures it mechanically, from the edits that attorneys are already making, and applies it automatically going forward.

05

The Founder's Journey: Research to Execution

This section is different from the rest of the white paper. It is not about the system. It is about the person who built it — the thinking, the wrong turns, the constraints, and the decisions that got made under pressure. If you are evaluating this work, this section is probably the most honest thing in the document.

Starting from Zero

The background here is not typical for an AI engineering assessment. BBA from NSU. No computer science degree. No formal software engineering courses. The way to describe the coding style accurately is: vibe coder. That is not false modesty — it is a precise description of someone who figures out what needs to be built, asks the right questions, uses the right tools, and ships.

What makes the background relevant is not the absence of technical training — it is the presence of something else: real experience with the messy gap between an idea and a working product. Running a BPO business means living inside operational complexity every day. It means knowing that technology decisions are never purely technical — they are always also budget decisions, timeline decisions, and risk decisions.

Before this assessment, there was already a working proof of concept for extracting structured data from PDFs and images to automate bank statement scoring for Merchant Cash Advance approvals. The domain was different, but the core problem — get reliable structured data out of unreliable documents — was identical. That is what made this assessment feel tractable.

What ChatGPT Taught Me

The first move before writing any code was to use AI to understand the problem space. ChatGPT was asked a direct question: given 12 hours, no existing infrastructure, and only Claude Code, what is the highest-probability strategy for this assessment?

The answer was clarifying in its specificity. It identified the core scoring criteria: messy document handling (the OCR story), grounded retrieval, evidence-aware generation, and — critically — the learning loop from edits, which ChatGPT flagged as the section most candidates would fail because they would implement a simple diff comparison rather than extracting reusable patterns. It also identified what not to build: no authentication, no frontend polish, no LangChain, no Kubernetes, no database infrastructure.

The research plan that followed from that conversation was step-by-step. First: understand what messy legal documents actually are in practice — not in the abstract, but the specific failure modes. Second: understand

how AI handles unclear inputs, which led to learning what OCR is and how it works at a mechanical level. Third: learn what happens to unclear or incomplete OCR output — which surfaced the concept of multi-layer OCR pipelines and probabilistic text reconstruction.

The initial lean stack identified through that research was: Tesseract for OCR, pymupdf for parsing, custom Python for chunking, sentence-transformers for embeddings, FAISS plus BM25 for search, Ollama for local LLM inference, FastAPI for the API, and Streamlit for a UI. This was a solid local stack — but it had a fundamental flaw. Ollama runs models on local compute, which is not practical outside a developer machine. A deployable system needed a real LLM API.

The Budget and Time Reality

Here is where the founder instinct overrode the engineer instinct. The dream execution plan — production-grade OCR, hosted vector database, streaming responses, Docker container, polished Streamlit UI — was fully mapped out. And then it was set aside entirely.

The constraints were absolute: 12 hours, no budget, and one tool — Claude Code with a Pro subscription. Nobody was paying for this project. Spending personal money on API keys to test an unpaid assessment was not a rational decision. The question became: what is the highest-value thing that can be built with exactly what is available?

"It is very easy to start building something and not bother about project deadlines or bloating the technology. In my case I think about shipping an MVP, get feedback and improve from there."

That instinct — ship the smallest thing that demonstrates the full loop correctly — drove every technical decision that followed.

The Decisions Made Under Pressure

Each architectural choice was a deliberate trade, not a default. Here is the honest account of what was chosen, what was rejected, and why:

FAISS over Pinecone or Weaviate

No infrastructure to spin up, no API key required, produces inspectable local files that a reviewer can examine directly. The trade-off — no horizontal scaling — is irrelevant for an MVP.

sentence-transformers over OpenAI embeddings

Runs locally at zero cost per query. The model quality is sufficient for legal document retrieval. The trade-off — lower embedding quality on very specialized legal terminology — is acceptable at MVP stage.

Local JSON over PostgreSQL

Zero setup, zero maintenance, fully inspectable. A reviewer can open `correction_memory.json` and read exactly what the system has learned. The trade-off — no query capability at scale — is a future problem.

Prompt injection over fine-tuning for the learning loop

Fine-tuning requires training infrastructure, labelled data, and compute budget. Prompt injection requires a JSON file and a string concatenation. The ROI comparison is not close under these constraints.

Mock mode with no API key required

The full pipeline — ingestion, OCR, chunking, FAISS indexing, retrieval — works without any API key. Draft generation falls back to a structured template populated with real metadata. This means the system can be evaluated end-to-end by anyone without needing credentials.

No frontend, no authentication, no Docker

Each of these would have consumed 1-2 hours of the 12-hour budget for features that do not appear in the assessment rubric. The Swagger UI provides interactive endpoint testing. FastAPI's auto-generated docs are themselves a demonstration of engineering quality.

There is one honest admission: not using an OpenRouter free API key to test the LLM generation path end-to-end was a laziness call. The mock mode works correctly, and the generation code is sound — but the system was not run with live LLM output due to the time pressure. That is a gap, and it is one that two weeks and \$200 of budget would close completely.

06

Architecture & Technology Stack

The system is built from seven independently testable modules, each with a single responsibility. The principle is: if any module needs to be replaced or upgraded, the rest of the pipeline does not change.

Layer	Tool / Library	Rationale
Language	Python 3.11+	Standard, fast, universal tooling
API	FastAPI	Auto-generated docs, type safety, async-ready
PDF Parsing	pymupdf	Fast, handles digital PDFs with high fidelity
OCR Fallback	Tesseract	Open source, robust, no API cost
Embeddings	sentence-transformers	Local execution, no per-query cost
Vector Search	FAISS	Lightweight, no server, inspectable files
LLM	Claude / OpenAI (pluggable)	Swappable via environment variable
NER	spaCy	Fast regex-first extraction, NER as enhancement
Storage	Local JSON	Inspectable, zero setup, zero overhead

The Eight API Endpoints

The FastAPI layer exposes eight endpoints that map directly to the pipeline stages:

Method	Endpoint	Purpose
POST	/api/v1/ingest	Upload PDF or image → extract, chunk, index
GET	/api/v1/documents	List all processed documents
GET	/api/v1/documents/{id}	Retrieve document metadata and chunks
POST	/api/v1/draft	Generate grounded Case Fact Summary Memo
GET	/api/v1/draft/{id}	Retrieve a previously generated draft
POST	/api/v1/edit	Submit operator edit → trigger learning loop
GET	/api/v1/memory	Inspect all accumulated correction patterns
GET	/api/v1/health	Health check

What Was Built vs. What Was Omitted — and Why

Engineering judgment is not just about what to build. It is equally about what not to build — and being able to explain why. Every item on the omitted list below was a conscious decision, not an oversight.

What Was Built	What Was Omitted
Full ingestion pipeline (3-tier OCR)	User authentication
FAISS retrieval with confidence weighting	Polished frontend UI
Evidence-only draft generation	Docker / containerisation
Correction memory learning loop	Async task queue (Celery/ARQ)
8 FastAPI endpoints with error handling	LangChain framework
14-point automated test suite	Fine-tuning / RLHF
End-to-end demo script (no server needed)	Kubernetes / cloud deploy
Evaluation script (4 metrics)	Streaming responses
Mock mode (no API key required)	Multi-user / multi-tenant support
README + architecture.md	Complex agent orchestration
5 bugs found and fixed with documentation	Neo4j / graph database

The rubric for this assessment rewards engineering judgment, grounded generation, and practical scope control — not feature count. Every item in the omitted column would have consumed time and budget without moving any of those three needles.

The Path to Production

The current system is an MVP in the true sense: it demonstrates the full loop correctly, every component is real and working, and it can be evaluated end-to-end without credentials. It is also an honest starting point, not a finished product.

The estimate for a production-ready version with a decent UI, hosted infrastructure, and live LLM integration: two weeks and \$200. Here is what that money and time buys:

Upgrade	Current MVP	Production Version
Vector Database	FAISS (local files)	pgvector or Weaviate (hosted)
OCR Quality	Tesseract (open source)	AWS Textract or Google Doc AI
LLM Integration	Mock mode / Claude API	Claude API with live key + caching
Storage	Local JSON files	PostgreSQL with proper schema
UI	Swagger auto-docs	React or Streamlit frontend
Deployment	Local only	Docker + cloud hosting (~\$30/mo)
Auth	None	API key or JWT per-operator
Learning loop	JSON file injection	Database-backed, per-firm preferences

The architectural decisions made in the MVP are not technical debt — they are deliberate placeholders. FAISS was chosen not because it is the best vector database, but because it is the best vector database for a zero-budget 12-hour build. The interfaces between modules are clean enough that swapping FAISS for pgvector is a one-file change. The same is true for every other component in the stack.

Conclusion

There is a version of this assessment where the candidate spends all 12 hours building an impressive-looking frontend over a weak retrieval system, with generation that produces fluent text and no citations. That submission looks polished. It does not work in a legal context.

The choice here was the opposite: build the unglamorous parts correctly. Make the OCR pipeline tiered and confidence-scored. Make retrieval penalise bad evidence rather than exclude it. Make generation refuse to fabricate by design. Make the learning loop extract reusable patterns rather than storing diffs. Build mock mode so the whole system can be evaluated without any credentials.

None of these decisions required a computer science degree. They required understanding the problem well enough to know which parts mattered, the business sense to know what could be deferred, and the discipline to stop building when the MVP was complete.

"The assessment is not asking for production-scale legal AI. It is asking whether you understand AI systems design under constraints."

That is what was built here. The system is real, the pipeline is end-to-end, the constraints were respected, and the decisions are documented. The next version — with a proper budget and two weeks — ships a product.

Author	Shihab Alam Jamshed
Email	alamjamshed051@gmail.com
Repository	github.com/jamshed051/pearson-litt-legal-ai
Assessment	Pearson Specter Litt — AI Engineer Take-Home
Date	May 2026
Tools Used	Claude Code (Pro subscription) — the only tool available